

Manipulação de Arquivos

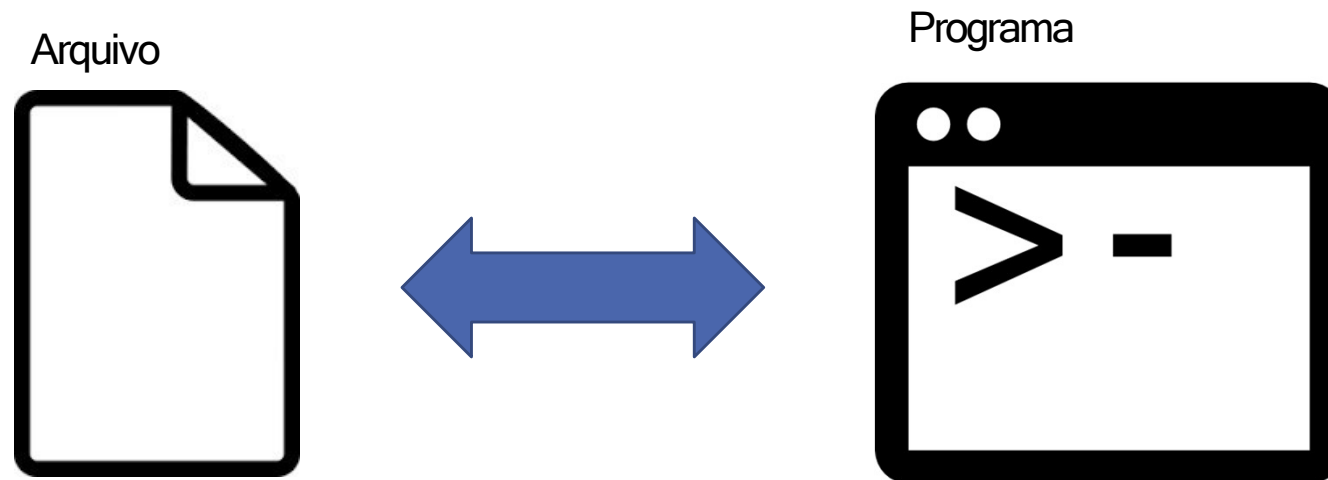
Diego Silva Caldeira Rocha

Sumário

- Arquivo
- Stream
- Buffer
- Tipos de Arquivo
- Manipulação de arquivos em Java
- Exercícios de treino

Arquivo

Arquivo é um conjunto de dados, dispostos de forma sequencial

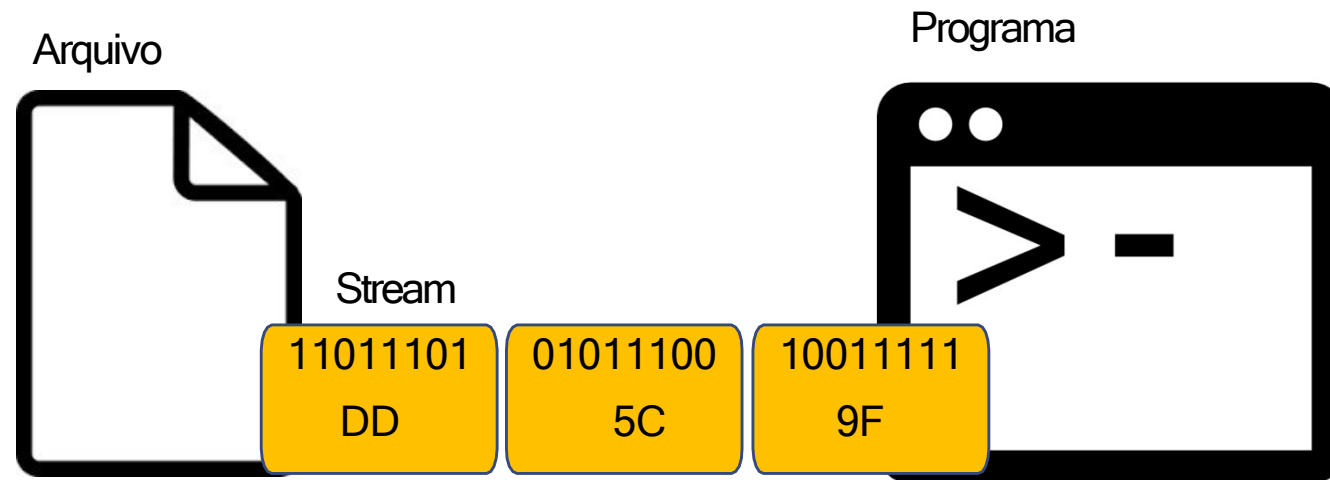


Arquivo

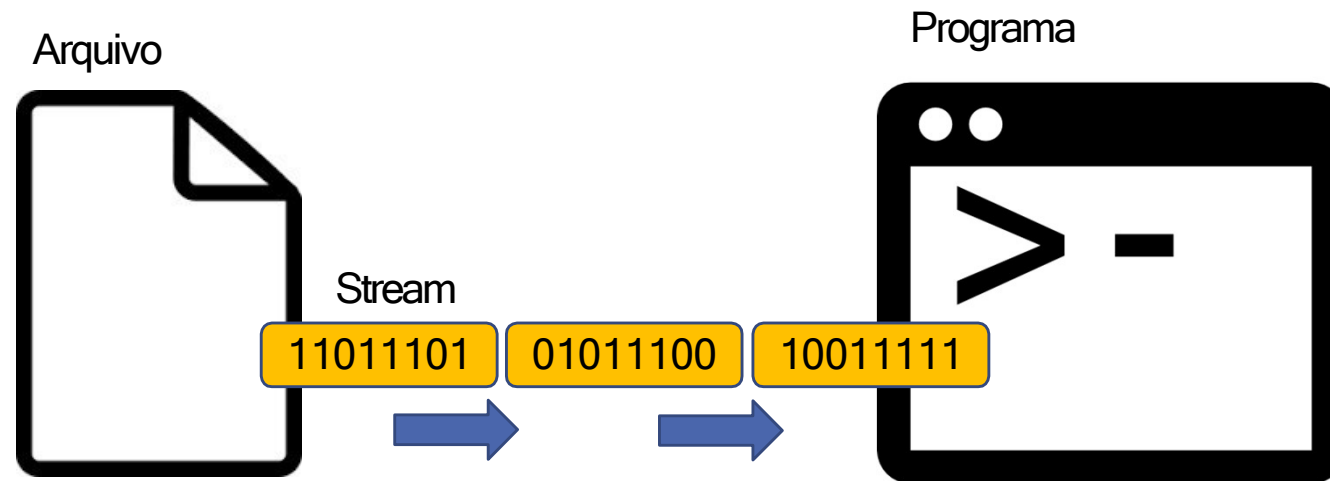
- O arquivo lógico é a maneira como os dados são organizados e vistos pelo usuário ou pelo programa, independentemente de como esses dados são armazenados fisicamente no disco.
- O arquivo físico refere-se à maneira como os dados são efetivamente armazenados no hardware, como um disco rígido ou SSD. Ele trata de detalhes como blocos de disco, endereçamento e localização física dos dados.

Stream

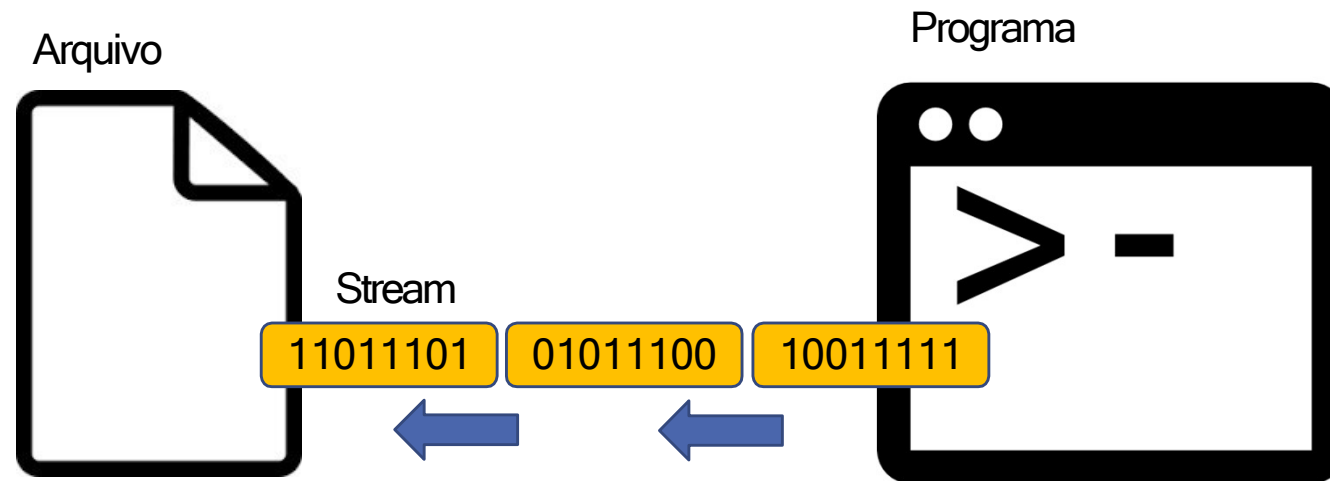
Leitura e escrita em um arquivo é feita por meio de um **stream**



Leitura de Dados

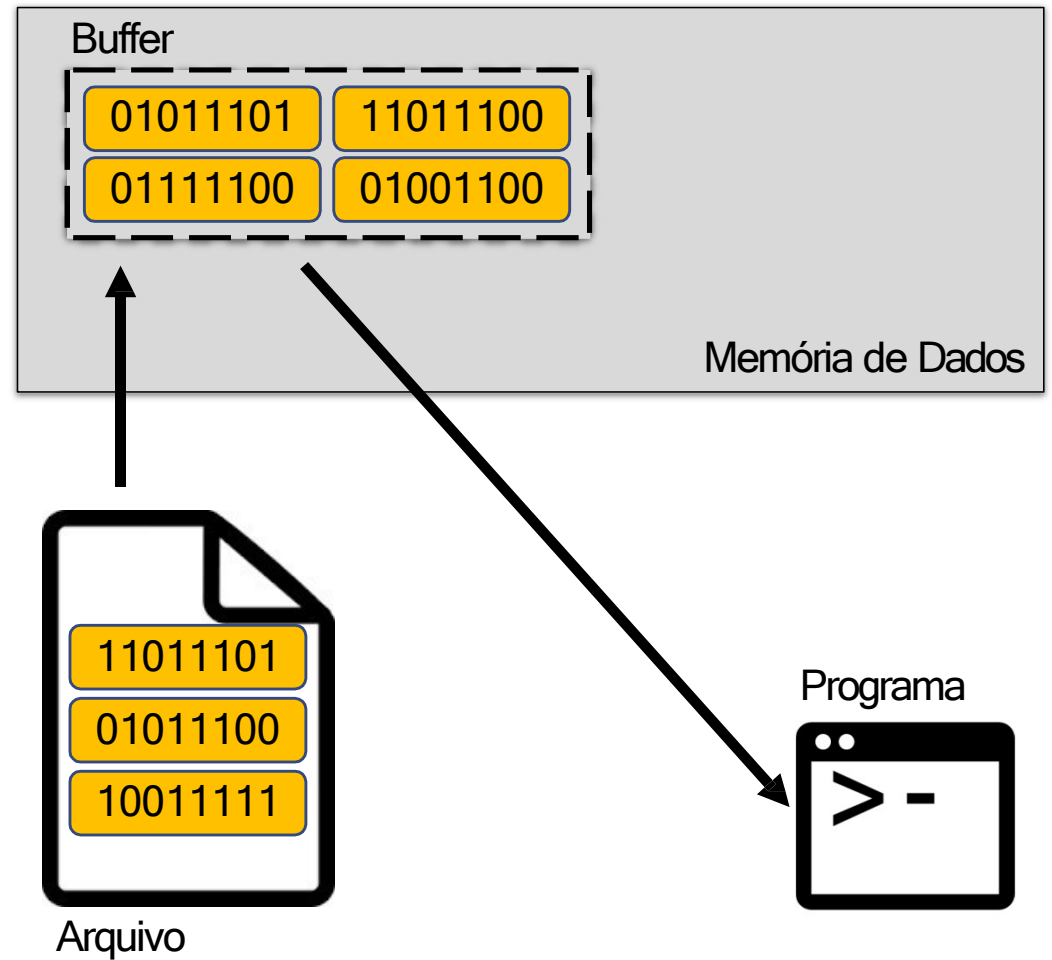


Escrita de Arquivos



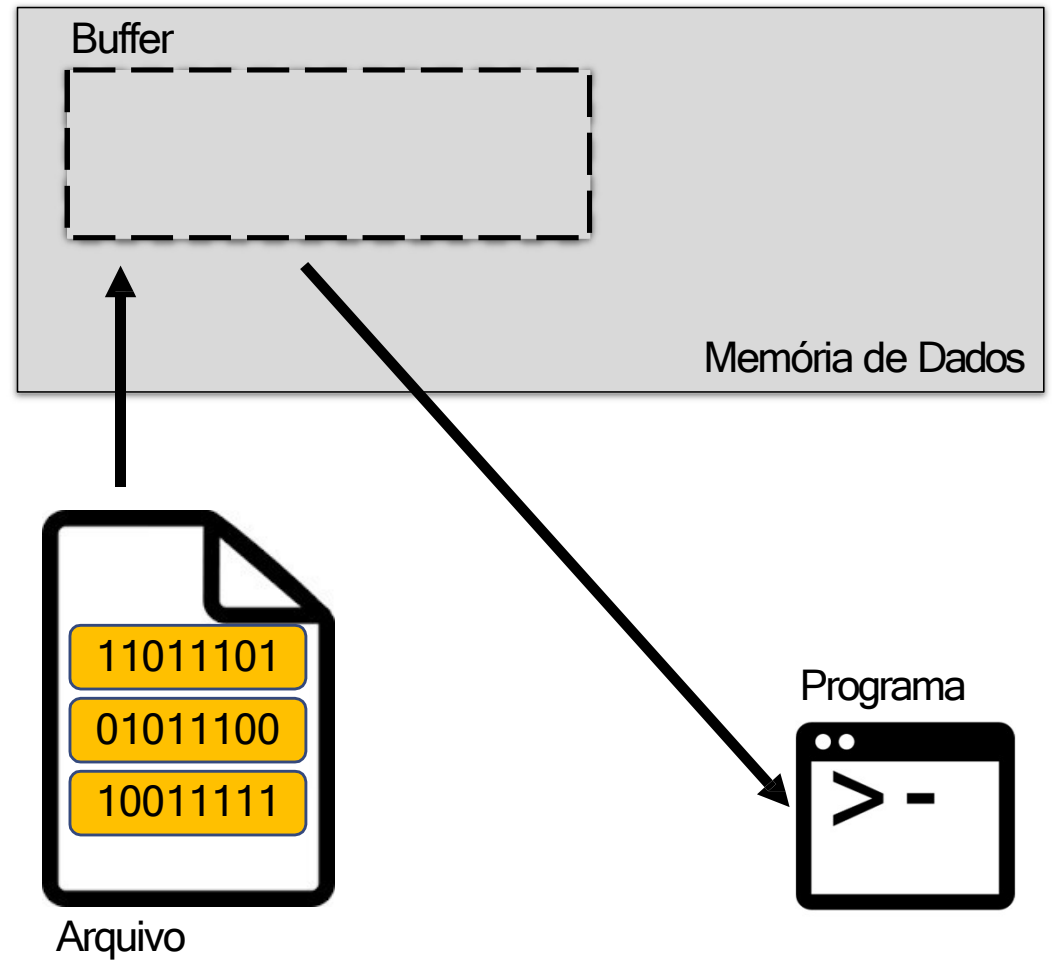
Buffer

Um buffer pode ser usado para acelerar a **leitura** e **escrita** de arquivos



Buffer para Leitura

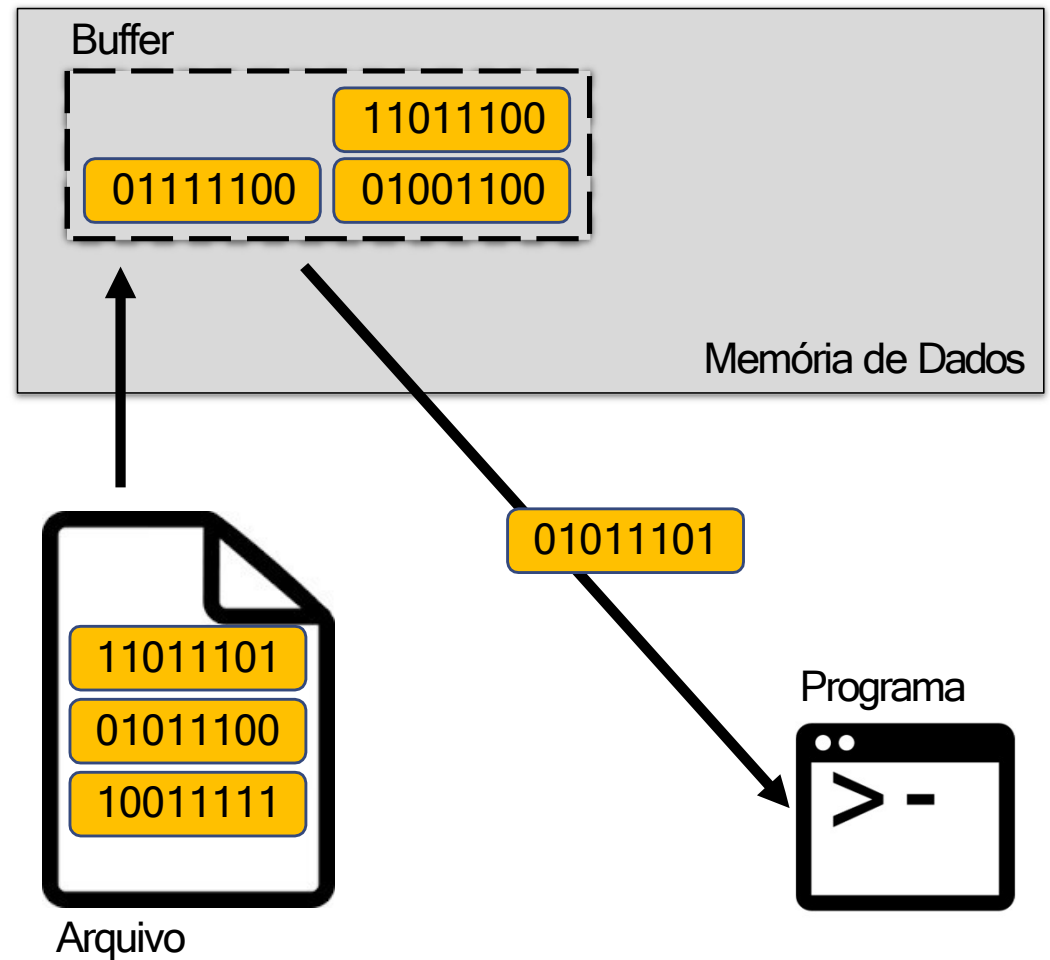
Situação inicial: buffer
vazio



Buffer Para Leitura

Programa faz operação de leitura

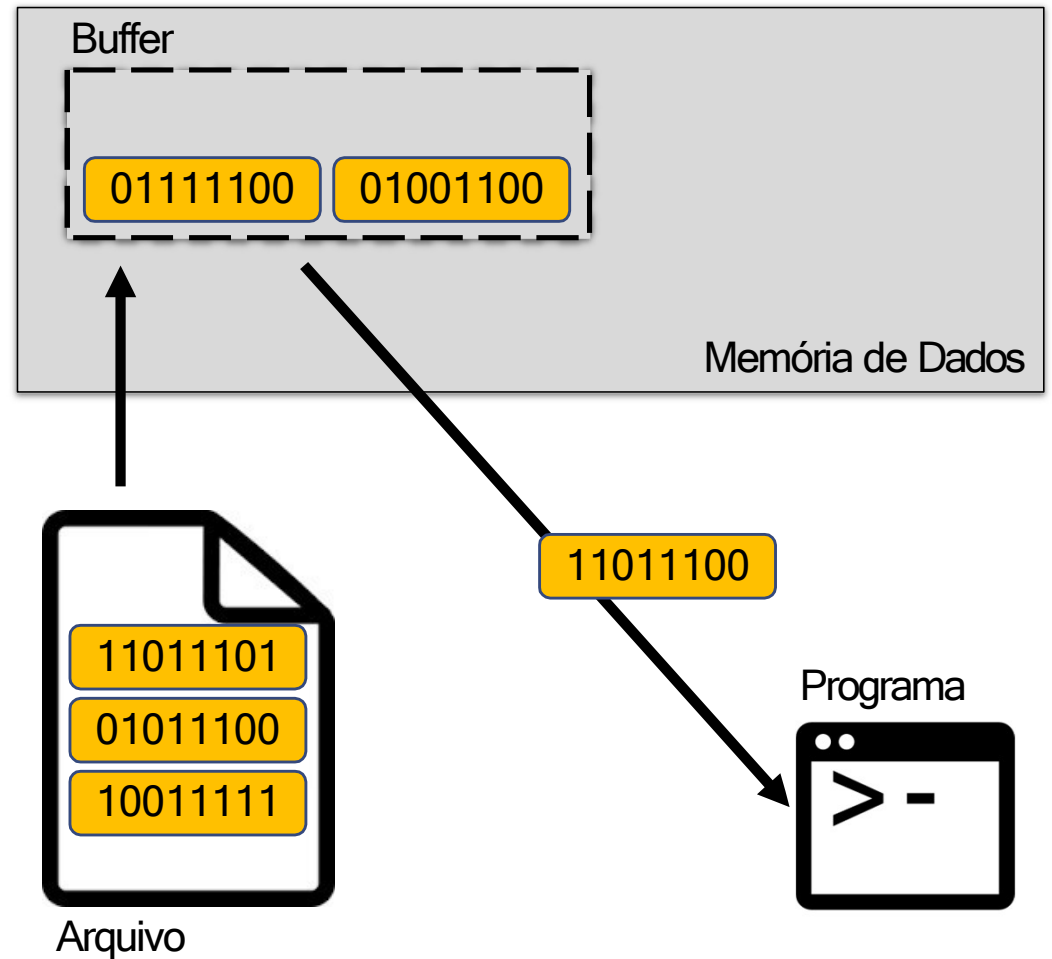
- Buffer está vazio, então dado é lido do arquivo para o buffer, até que ele fique cheio (ou que o arquivo acabe)
- Dado requisitado na leitura é enviado ao programa a partir do buffer (usando o stream que está conectado ao arquivo)



Buffer Para Leitura

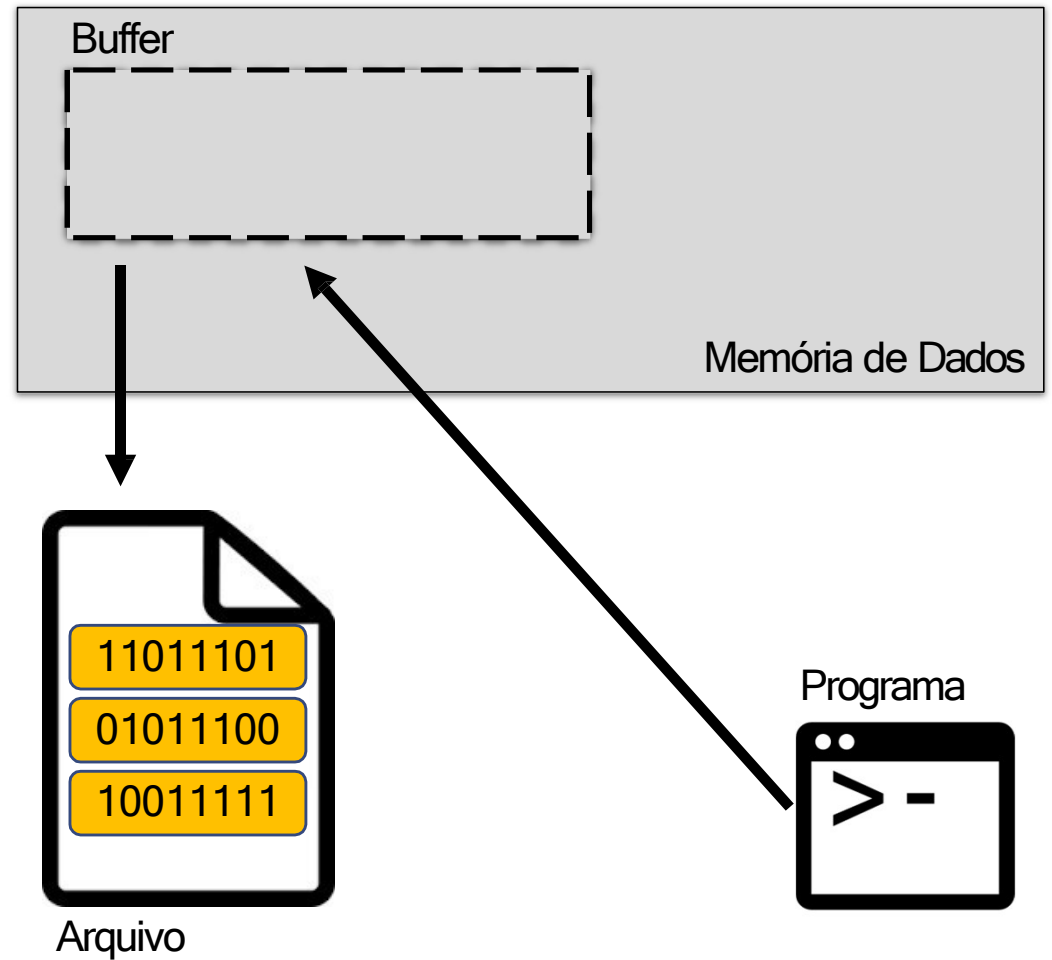
Programa faz operação de leitura

- Buffer está vazio, então dado é lido do arquivo para o buffer, até que ele fique cheio (ou que o arquivo acabe)
- Dado requisitado na leitura é enviado ao programa a partir do buffer (usando o stream que está conectado ao arquivo)



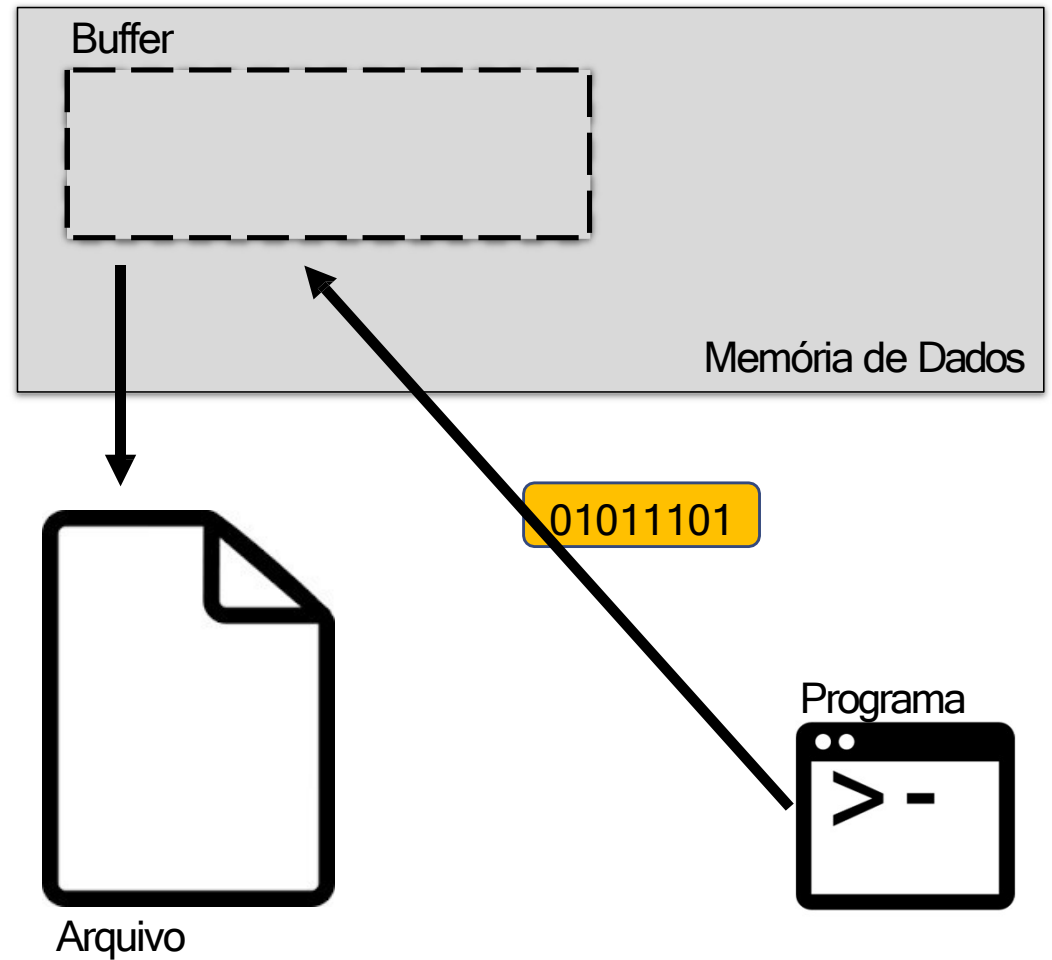
Buffer para Escrita

Situação inicial: buffer
vazio



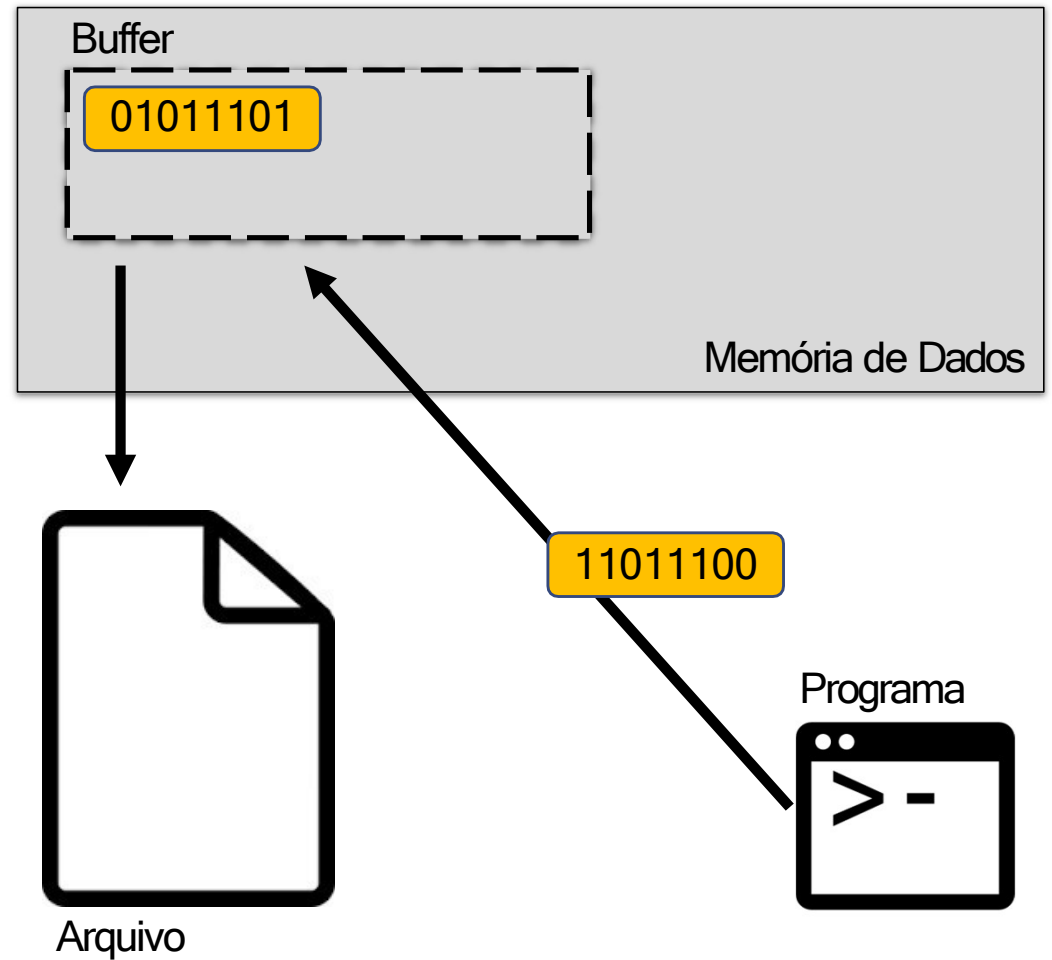
Buffer para Escrita

Escrita vai sendo feita no
buffer



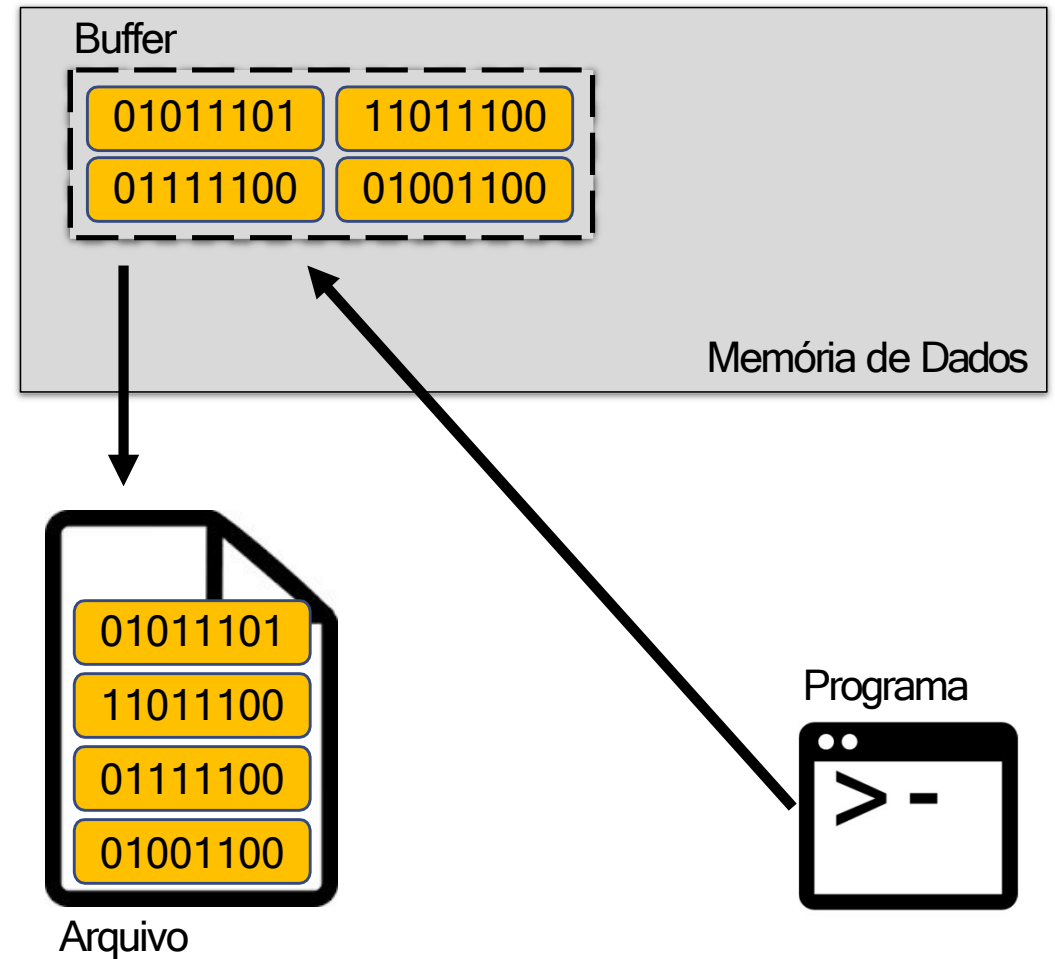
Buffer para Escrita

Escrita vai sendo feita no
buffer



Buffer para Escrita

Quando o buffer enche ele é automaticamente descarregado no arquivo

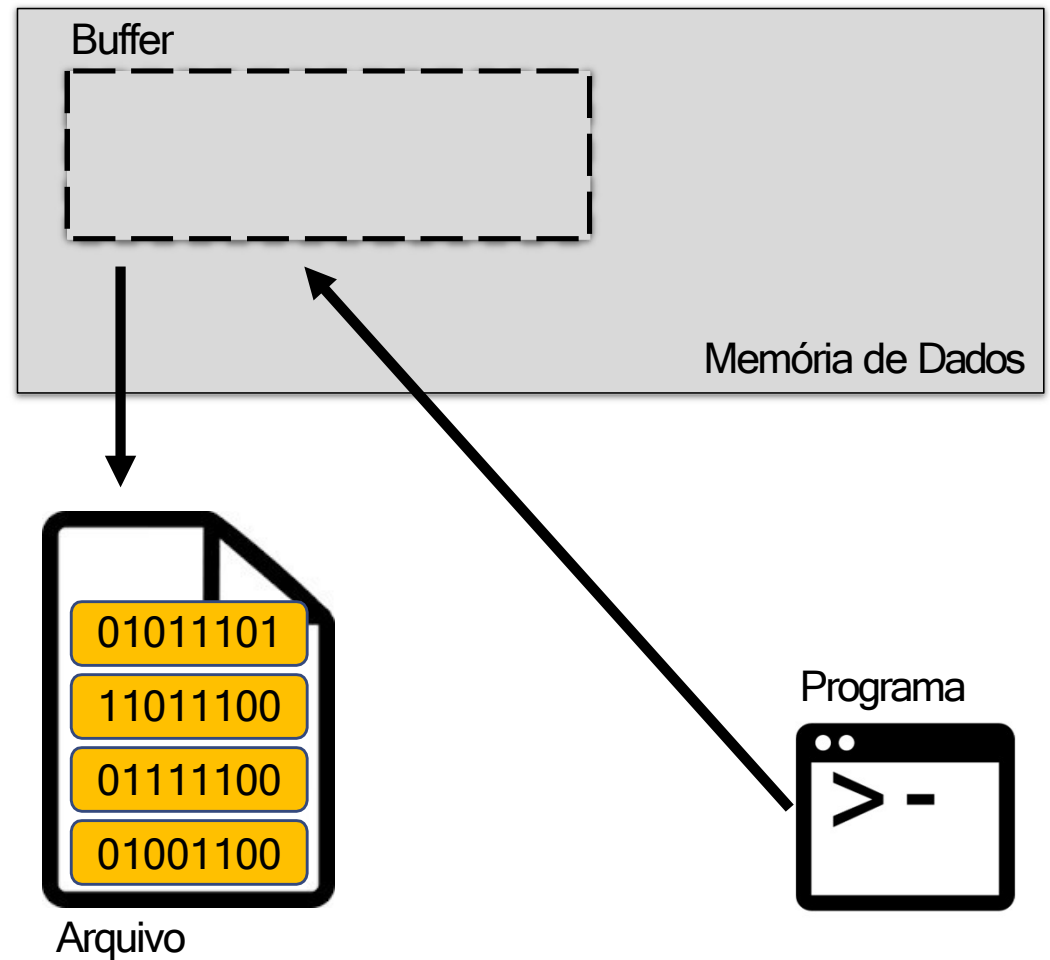


Buffer para Escrita

Quando o buffer enche ele é automaticamente descarregado no arquivo

Depois é esvaziado para aguardar novas escritas...

Essa operação se chama **flush**



Tipos de Arquivos

- Arquivo de Sistema
 - DIRETÓRIOS
- Arquivo Especiais
 - dispositivos de E/S como arquivos especiais:
 - DE BLOCO: modelam dispositivos constituídos por blocos de Tamanho fixo endereçáveis aleatoriamente (discos).
 - DE CARACTERES: modelam dispositivos constituídos por cadeias de caracteres (ex. terminais, teclados, impressoras, portas seriais, placas de rede, etc).
- Arquivo Regulares
 - Binário (programa executável (.exe), objeto (.obj), biblioteca (.bib), imagem, som, etc.
 - Texto ou ASCII (.txt, programa fonte(.c, .h, .cpp, .sh), etc.

Arquivo Texto (ASCII)

Conteúdo é apenas texto



Arquivo Binário

Conteúdo é binário



Manipulação de Arquivo em Java (Escrita)

```
import java.io.*;

public class Main {

    public static void main(String[] args) {

        try {

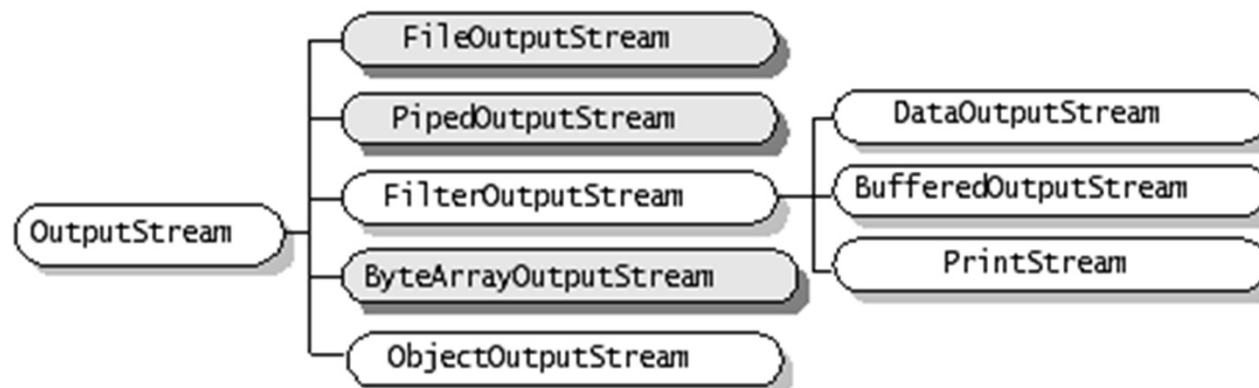
            FileOutputStream wr = new FileOutputStream("arquivo.txt");
            DataOutputStream dos = new DataOutputStream(wr);
            dos.write(56); //em byte: 38 42 37 99
            dos.writeFloat(45.9f); //em byte: 00 06 4B C3
            dos.writeUTF("Aliança");//byte 41 6C 69 61 6E C3 A7 61
            dos.close(); //https://hexed.it/

        } catch (Exception e){
            System.out.println(e.getMessage());
        }
    }
}
```

Streams

Byte Streams

Stream de Saída → OutputStream



Streams

A classe *OutputStream* declara entre outros métodos os seguintes:

```
public abstract void write(int b) throws IOException
```

Esse método realiza a escrita de `b` como um byte, isto é, apenas os 8 bits menos significativos do inteiro `b` serão armazenados.

```
public void write(byte[] buff, int desloc, int cont) throws IOException
```

Esse método realiza a escrita de `cont` bytes. Os bytes devem estar armazenados no vetor `buff` a partir da posição `desloc`.

```
public void flush() throws IOException
```

Esse método "força" a escrita imediata no destino de quaisquer bytes enviados para o *stream*. Caso o destino seja outro *stream* ele também será obrigado a escrever seus bytes imediatamente (seu método `flush()` será invocado), em outras palavras, apenas uma única chamada de `flush()` é necessária para uma série de *streams* encadeados.

```
public void close() throws IOException
```

Esse método fecha o *stream*. Ele deve ser invocado de modo a liberar todos os recursos associados ao *stream*. Uma vez fechado o *stream* não deve ser utilizado e o fechamento de um *stream* já fechado não tem nenhum efeito.

Manipulação de Arquivo em Java (leitura)

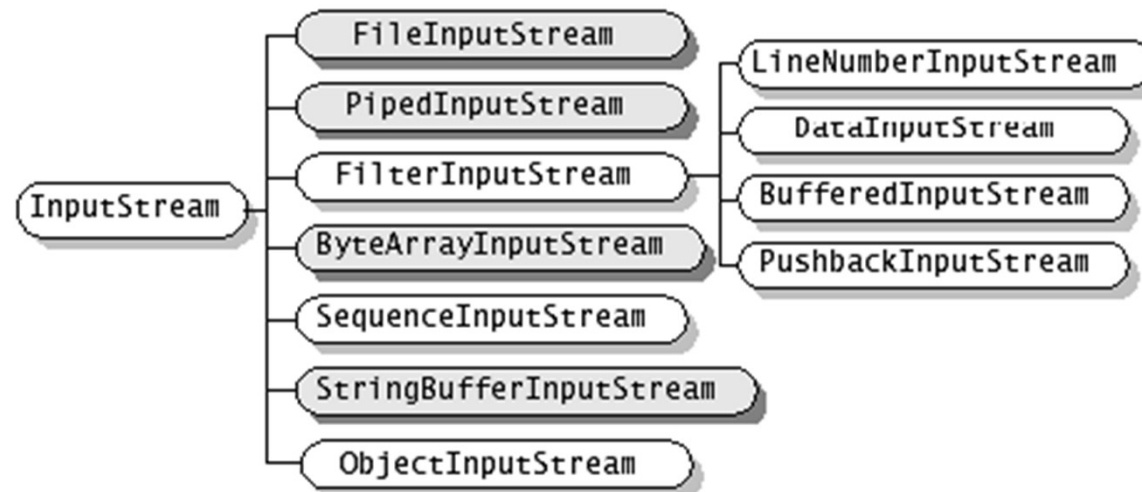
```
import java.io.*;

public class Main {
    public static void main(String[] args) {
        try {
            FileReader fr=new FileReader("arquivo.txt");
            BufferedReader reader = new BufferedReader(fr);
            String linha;
            while ((linha = reader.readLine()) != null) {
                System.out.println(linha);
            }
            reader.close();
        } catch (Exception e){
            System.out.println(e.getMessage());
        }
    }
}
```

Streams

Byte Streams

Stream de Entrada → InputStream



Streams

A classe *InputStream* declara entre outros métodos os seguintes:

```
public abstract int read() throws IOException
```

Esse método realiza a leitura de um único byte e o retorna como um inteiro no intervalo de 0 a 255. Caso não haja byte disponível para leitura devido ao fim do *stream* (por exemplo, quando o fim de arquivo for alcançado) o valor -1 será retornado.

```
public int read(byte[] buff, int deslo, int cont) throws IOException
```

Esse método realiza a leitura de até `cont` bytes. Os bytes são armazenados no vetor `buff` a partir da posição `deslo`. O método retorna o número de bytes efetivamente lidos ou -1 caso o fim de *stream* seja encontrado.

```
public long skip(long cont) throws IOException
```

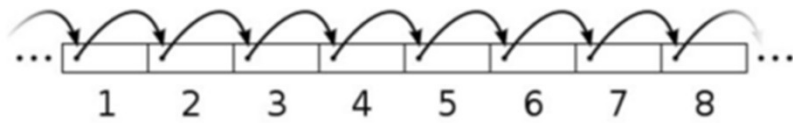
Esse método "salta" a leitura de até `cont` bytes. O método retorna o número de bytes efetivamente "saltados".

```
public void close() throws IOException
```

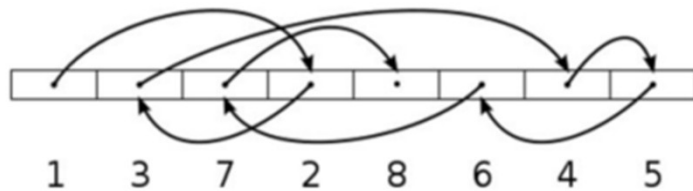
Esse método fecha o stream. Ele deve ser invocado de modo a liberar todos os recursos associados ao stream. Uma vez fechado o stream não deve ser utilizado e o fechamento de um stream já fechado não tem nenhum efeito.

Acesso Sequencial e Aleatório

Acesso Sequencial



Acesso Aleatório



Manipulação de Arquivo em Java (leitura e escrita)

- Classe Random Access File
- Permite o acesso aleatório de leitura e escrita a arquivos
- Manipula arquivos em modo binário
- Considera que cada arquivo é um array de bytes indexado por um cabeçote (ponteiro cursor).

Manipulação de Arquivo em Java (leitura e escrita)

- Construtor RAF
 - Recebem como parâmetro uma referência para um arquivo e um modo de operação indicando leitura ou escrita
 - `RandomAccessFile(File file, String mode)`
 - `RandomAccessFile(String name, String mode)`
- "r", apenas leitura.
- "rw", leitura/escrita. Se o arquivo não existir, o mesmo é criado.

Manipulação de Arquivo em Java (leitura escrita)

```
import java.io.*;

public class Main {

    public static void main(String[] args) {

        try {

            RandomAccessFile raf = new
RandomAccessFile("arquivo.txt", "rw");

            raf.writeInt(1);
            raf.writeDouble(5.3);
            raf.writeChar('X');
            raf.writeBoolean(true);
            raf.writeBytes("Algoritmos");
            raf.seek(0); //Retornando o cabecote para a posicao 0
            System.out.println(raf.readInt()); //imprimindo o inteiro
```

```
raf.seek(12); //Setando o cabecote na posicao 12 (do caractere, //12 = 4 do
int + 8 do double)

            System.out.println(raf.readChar());

            raf.seek(12); //Setando o cabecote novamente na posicao 12

            raf.writeChar('@'); //Substituindo o caractere

            raf.seek(12);

            System.out.println(raf.readChar());

            raf.close();

        }

        catch (Exception e){

            System.out.println(e.getMessage());

        }

    }

}
```

Object Byte Streams e Serialização de Objetos

- Os Object Streams – `ObjectInputStream` e `ObjectOutputStream` – permitem que uma aplicação leia e escreva uma cadeia de objetos de/em um stream, além de permitir o armazenamento dos tipos primitivos, strings e vetores
- Ao se armazenar um objeto em um stream, todos os bytes representando o objeto – incluindo todos os objetos que ele referencia – são armazenados no stream
- Este processo de transformação de um objeto em um fluxo ("stream") de bytes é denominado de serialização.
- Para que os objetos de uma classe possam ser serializados, a classe deve implementar a interface de marcação (interface vazia) `Serializable`

Exemplo serialização em Java

```
import java.io.*;

class Pessoa implements Serializable
{
    private String nome;
    private int idade;

    // Construtor da classe
    Pessoa(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    @Override
    public String toString() {
        return "Pessoa{" +
            "nome=" + nome + "\" +
            ", idade=" + idade +
            '"';
    }
}
```

Serelização

```
public static void main(String[] args) {  
    // Criando uma lista de pessoas  
    List<Pessoa> listaPessoas = new ArrayList<>();  
    listaPessoas.add(new Pessoa("João", 30));  
    listaPessoas.add(new Pessoa("Maria", 25));  
    listaPessoas.add(new Pessoa("Carlos", 40));  
    try {  
        ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("arquivo.txt"));  
        oos.writeObject(listaPessoas);  
        System.out.println("Lista de objetos serializada com sucesso para o arquivo");  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```


Desserialização em Java

```
public static void main(String[] args) {  
    // Desserializando a lista de pessoas  
    do arquivo  
    try{  
        ObjectInputStream ois = new  
ObjectInputStream(new  
FileInputStream("arquivo.txt"));  
        // Lendo o objeto serializado e  
fazendo o cast para List<Pessoa>  
        List<Pessoa>  
listaPessoasDesserializada =  
(List<Pessoa>) ois.readObject();  
        System.out.println("Lista de  
objetos desserializada com sucesso!");  
    }
```

```
        // Iterando sobre a lista e mostrando os  
dados de cada pessoa  
        for (Pessoa pessoa :  
listaPessoasDesserializada) {  
            System.out.println(pessoa);  
        }  
  
    } catch (IOException |  
ClassNotFoundException e) {  
        e.printStackTrace();  
    }  
}
```

Exercícios (não precisa entregar)

- 1) Implementar um stream de entrada de dados utilizando a classe ***FileInputStream*** e seu método ***read*** para contar o número de bytes de um arquivo
- 2) Implementar um programa em java que exemplifique o uso de ***DataByteStreams*** para realizar a escrita e a leitura de um vetor números reais (declarados como double) em um arquivo.

Próxima Aula

- Armazenamento em Memória Secundária